Namespaces, Assemblies, Versioning, Class Libraries, and NuGet

Guide covering namespaces, assemblies and manifests, versioning (private vs shared, strong names), class libraries (.NET Standard vs .NET Core), and settings / NuGet packages.

1. Namespaces and Their Purpose

1.1 What a namespace is

A **namespace** is a logical container for related types (classes, structs, interfaces, enums, delegates).

It does **not**:

- create a file or folder by itself
- control how code is deployed
- equal an assembly

It **does**:

- group types into a clear hierarchy
- avoid name collisions (`MyApp.Models.User` vs `External.Crm.User`)
- express ownership and domain (`Company.Product.Feature`)

Mental model:

```
Logical world (namespaces)          Physical world (assemblies)
──────────────────────────          ──────────────────────────────
Company.Billing.Invoices   →    Billing.dll
Company.Billing.Payments   →    Billing.dll  (same assembly OK)
Company.Shipping.Routes    →    Shipping.dll
```

Namespaces organize **names**. Assemblies organize **deployment**.

1.2 Why namespaces exist

Without namespaces, every type name must be globally unique across the whole program and all libraries you reference. That does not scale.

With namespaces:

- two libraries can both define `Order`
- you disambiguate with the full name or an alias
- APIs stay readable: related types live under one prefix

Example collision:

```
// Your code
namespace Acme.Shop
{
    public class Order { }
}

// Third-party CRM
namespace Contoso.Crm
{
    public class Order { }
}

// Usage with full names
var shopOrder = new Acme.Shop.Order();
var crmOrder = new Contoso.Crm.Order();
```

1.3 Declaring and using namespaces

Classic block form

```
namespace Acme.Shop.Orders
{
    public class OrderService
    {
        public void Place(Order order) { }
    }

    public class Order { }
}
```

File-scoped namespace (C# 10+)

One namespace for the whole file — less nesting:

```
namespace Acme.Shop.Orders;

public class OrderService
{
    public void Place(Order order) { }
}
```

Nested namespaces

```
namespace Acme
{
    namespace Shop
    {
        namespace Orders
        {
            public class Order { }
        }
    }
}

// Same as: namespace Acme.Shop.Orders
```

`using` directives

```
using System;
using System.Collections.Generic;
using Acme.Shop.Orders;

// Alias when names collide
using ShopOrder = Acme.Shop.Order;
using CrmOrder = Contoso.Crm.Order;

ShopOrder a = new();
CrmOrder b = new();
```

Useful `using` forms:

Form	Purpose
`using System.IO;`	Import types from a namespace
`using static System.Math;`	Import static members (`Sqrt` , `PI`)
`using Json = System.Text.Json;`	Namespace alias
`global using System;`	Project-wide import (often in `GlobalUsings.cs`)

`using static` example

`using static` imports the **static members** of a type so you can call them without the type name:

```
using static System.Math;

public class Circle
{
    public double Radius { get; set; }

    // Without using static: Math.PI, Math.Pow, Math.Round
    // With using static:    PI, Pow, Round
    public double Area
    {
        get { return Round(PI * Pow(Radius, 2), 2); }
    }

    public double Circumference
    {
        get { return Round(2 * PI * Radius, 2); }
    }
}

// Usage
public static class Demo
{
    public static void Main()
    {
        Circle c = new Circle();
        c.Radius = 5;
        Console.WriteLine(c.Area);           // 78.54
        Console.WriteLine(c.Circumference);  // 31.42
    }
}
```

Another common case — `using static` with your own helpers:

```
using static Acme.Shop.Pricing.PriceRules;

namespace Acme.Shop.Pricing;

public static class PriceRules
{
    public static decimal WithTax(decimal amount, decimal rate) => amount * (1 + rate);
    public static decimal Discount(decimal amount, decimal percent) => amount * (1 - pe
}

public class Checkout
{
    public decimal Total(decimal subtotal)
    {
        var afterDiscount = Discount(subtotal, 0.10m);
        return WithTax(afterDiscount, 0.20m);
    }
}
```

Use `using static` when a type is mostly a bag of helpers (`Math`, `Console` is usually *not* worth it — readability suffers if overused).

`global using` example

`global using` applies the import to **every file in the project**, so you do not repeat it in each file.

`GlobalUsings.cs` (any one file in the project is enough):

```
global using System;
global using System.Collections.Generic;
global using System.Linq;
global using System.Threading.Tasks;
```

Then in another file you can use those types with no local `using`:

```
// No "using System;" needed here — covered by global using
namespace Acme.Shop.Orders;

public class OrderService
{
    public List<string> GetOpenOrderIds()
    {
        // List<> from System.Collections.Generic
        // Enumerable methods from System.Linq
        return new List<string> { "A-1", "A-2" }
            .Where(id => id.StartsWith("A"))
            .ToList();
    }

    public async Task DelayDemoAsync()
    {
        // Task from System.Threading.Tasks
        await Task.Delay(100);
    }
}
```

`ImplicitUsings` in the `.csproj`

```
<PropertyGroup>
  <ImplicitUsings>enable</ImplicitUsings>
</PropertyGroup>
```

This is an **SDK feature** (from .NET 6 onward), not a C# language keyword. When enabled, the build generates a file under `obj/` with `global using` directives for a default set of namespaces that depends on which **project SDK** you use.

Typical generated file path:

```
obj/Debug/net8.0/<ProjectName>.GlobalUsings.g.cs
```

Example of what that generated file looks like for a console / class library project:

```
// <auto-generated/>
global using global::System;
global using global::System.Collections.Generic;
global using global::System.IO;
global using global::System.Linq;
global using global::System.Net.Http;
global using global::System.Threading;
global using global::System.Threading.Tasks;
```

You do **not** edit that file by hand — it is regenerated on build. New project templates usually set `ImplicitUsings` to `enable` already. Older projects often have it off unless you add the property.

Which namespaces are imported?

The set depends on the `Sdk="..."` attribute at the top of the `.csproj` .

`Microsoft.NET.Sdk` (console apps, class libraries, most non-web projects):

Implicit global using
`System`
`System.Collections.Generic`
`System.IO`
`System.Linq`
`System.Net.Http`
`System.Threading`
`System.Threading.Tasks`

`Microsoft.NET.Sdk.Web` (ASP.NET Core) — everything from `Microsoft.NET.Sdk` , **plus**:

Extra implicit global using
`System.Net.Http.Json`
`Microsoft.AspNetCore.Builder`
`Microsoft.AspNetCore.Hosting`
`Microsoft.AspNetCore.Http`

Extra implicit global using
`Microsoft.AspNetCore.Routing`
`Microsoft.Extensions.Configuration`
`Microsoft.Extensions.DependencyInjection`
`Microsoft.Extensions.Hosting`
`Microsoft.Extensions.Logging`

`Microsoft.NET.Sdk.Worker` (Worker Service) — everything from `Microsoft.NET.Sdk`, **plus**:

Extra implicit global using
`Microsoft.Extensions.Configuration`
`Microsoft.Extensions.DependencyInjection`
`Microsoft.Extensions.Hosting`
`Microsoft.Extensions.Logging`

`Microsoft.NET.Sdk.WindowsDesktop`:

- **WinForms:** base SDK usings + `System.Drawing`, `System.Windows.Forms`
- **WPF:** base SDK usings, but `System.IO` and `System.Net.Http` are removed (to avoid name clashes with WPF types)

Customize: add or remove individual usings

Keep `ImplicitUsings` on, but tweak the list in the `.csproj`:

```
<ItemGroup>
  <!-- Add your own as an implicit global using -->
  <Using Include="Acme.Shop.Orders" />

  <!-- Remove one the SDK added -->
  <Using Remove="System.Net.Http" />
</ItemGroup>
```

That is equivalent to writing `global using Acme.Shop.Orders;` / not importing `System.Net.Http`.

What is the purpose of `ImplicitUsings` ?

Purpose of `enable` : less boilerplate.
Without it, almost every file starts with the same lines:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
// ...
```

With `<ImplicitUsings>enable</ImplicitUsings>` , the SDK writes those as project-wide `global using` s for you. So in `Program.cs` you can use `List<T>` , `Console` , `Task` , LINQ, etc. even if the file has **no** `using` lines at the top.

Purpose of `disable` : turn that convenience **off**.
It does **not** give you new APIs, packages, or settings. It only means:

- the SDK will **not** generate those automatic global usings
- types from `System` , `System.Linq` , etc. are **not** imported unless **you** write `using` / `global using` yourself

Example — same code, different setting:

```
// Program.cs
var numbers = new List<int> { 1, 2, 3 };  // List<> is in System.Collections.Generic
Console.WriteLine(numbers.Count);
```

Setting	Does this compile as-is?
`ImplicitUsings = enable`	Yes — SDK already imported `System` and `System.Collections.Generic`
`ImplicitUsings = disable`	No — you must add `using System;` and `using System.Collections.Generic;` (or your own `global using`)

So:

- **`enable`** = “import the common stuff for me”
- **`disable`** = “don’t import anything for me; I’ll manage usings myself”

Why would anyone disable it?

1. Prefer explicit usings so each file shows exactly what it depends on
2. Avoid name clashes (two types with the same short name from different namespaces)
3. Teaching / demos where you want students to see every `using`
4. Migrating old code that already has careful, manual imports

```
<PropertyGroup>
  <ImplicitUsings>disable</ImplicitUsings>
</PropertyGroup>
```

After that, only explicit `using` / `global using` in your source files count.

`ImplicitUsings` vs hand-written `global using`

	`ImplicitUsings`	Your `global using` in `GlobalUsings.cs`
Who adds it?	SDK / MSBuild	You
Where does it live?	Generated under `obj/`	Source file you commit
Default set?	Yes, by project SDK	Only what you write
Good for	Boilerplate BCL / framework namespaces	Project-wide domain imports you choose

They work together: enable `ImplicitUsings` for the SDK defaults, and add a `GlobalUsings.cs` only for extra namespaces you want everywhere.

Prefer implicit / global usings for truly universal imports. Keep feature-specific namespaces (`Acme.Shop.Payments`) as normal per-file `using` directives so intent stays clear.

1.4 Namespace vs folder vs project

Concept	Role
Folder	How files are stored on disk
Namespace	How types are named in code

Concept	Role
Project / assembly	What gets compiled and shipped

Convention: folder path mirrors namespace.

```
src/Acme.Shop/
  Orders/
    Order.cs          → namespace Acme.Shop.Orders
    OrderService.cs   → namespace Acme.Shop.Orders
  Payments/
    Payment.cs        → namespace Acme.Shop.Payments
```

This is a convention, not a compiler rule. Types can live in any folder; the `namespace` declaration wins.

1.5 Purpose in real projects

Namespaces communicate **boundaries**:

```
Acme.Shop                  → product root
Acme.Shop.Domain           → entities, value objects
Acme.Shop.Application      → use cases / services
Acme.Shop.Infrastructure   → EF, HTTP clients, files
Acme.Shop.Api              → controllers / endpoints
```

Benefits:

1. **Discoverability** — find related types by prefix
2. **Encapsulation by convention** — keep infrastructure types out of domain usings
3. **Safe reuse** — publish libraries without polluting the global name space
4. **Team clarity** — ownership of areas is visible in the name

1.6 Best practices

- Prefer `Company.Product.Area` over short generic names (`Utils` , `Helpers` alone).
- Keep namespaces stable; renaming is a breaking change for consumers.
- Align namespaces with feature/domain, not with layer-only names if that creates confusion.
- Avoid deep trees (`A.B.C.D.E.F`) unless the product really needs that depth.

- Use file-scoped namespaces in modern C# for flatter files.
- Prefer `global using` for truly universal imports (`System` , `System.Linq`), not for every project namespace.

1.7 Quick check

Question	Answer
Does a namespace create a DLL?	No
Can one assembly contain many namespaces?	Yes
Can one namespace span many assemblies?	Yes (possible, usually avoided)
What solves name collisions?	Full name or `using` alias
What should mirror folders?	Namespace (by convention)

Takeaway: Namespaces are the naming system of .NET types. They organize code logically; assemblies organize it physically for deployment.

2. Assemblies and Manifest

2.1 What an assembly is

An **assembly** is the basic unit of deployment, versioning, and security identity in .NET.

In practice it is usually:

- a `.dll` (class library or shared component), or
- an `.exe` / runnable app output (still built around assemblies)

An assembly contains:

1. **IL (Intermediate Language)** — compiled code
2. **Metadata** — type definitions, method signatures, references
3. **Manifest** — identity and dependency information for the assembly itself
4. Optionally **resources** — strings, images, embedded files

Conceptual picture:

```
MyApp.dll (assembly)
├── Manifest          ← "who I am" + "what I need"
├── Type metadata     ← classes, methods, fields
├── IL code           ← executable instructions
└── Resources         ← optional embedded data
```

2.2 Assembly vs namespace vs module

Term	Meaning
Namespace	Logical name grouping for types
Assembly	Physical deployable unit (DLL/EXE + identity)
Module	Single file that can be part of a multi-file assembly (rare today)

Most modern .NET projects produce **one assembly per project** (single-file assembly).

```
Acme.Shop.csproj  →  Acme.Shop.dll
Acme.Shop.Api.csproj → Acme.Shop.Api.dll
```

What is a module? (multi-file assemblies)

In .NET terms, a **module** is one compiled file (historically a `.netmodule`) that contains IL + type metadata, but **does not have to be a complete assembly by itself**.

- A normal `.dll` / `.exe` is usually **one module that is also the whole assembly** (and that module holds the manifest).
- A **multi-file assembly** is rarer: several modules are linked into **one** assembly identity. Only one of those files holds the **manifest**; the others are netmodules that belong to that same assembly.

Conceptual picture:

```
Single-file assembly (normal today)
─────────────────────────────────────

Acme.Shop.dll
  ├── Manifest
  ├── Types / IL
  └── (this file IS the assembly)


Multi-file assembly (rare / legacy)
─────────────────────────────────────

Acme.Shop.dll          ← primary module (has the manifest)
  └── Manifest says: "I also own Core.netmodule and Utils.netmodule"

Core.netmodule         ← secondary module (IL + types, no assembly manifest)
Utils.netmodule        ← secondary module
```

From the outside, consumers still reference **one assembly** (`Acme.Shop`), not three independent libraries. `internal` visibility is shared across all modules of that assembly.

Illustrative compile steps (classic `csc` style — not how normal `dotnet build` works):

```
# 1) Compile pieces into modules (no assembly yet)
csc /t:module Core.cs          # → Core.netmodule
csc /t:module Utils.cs         # → Utils.netmodule

# 2) Compile the main file and link modules into ONE assembly
csc /t:library /out:Acme.Shop.dll Shop.cs /addmodule:Core.netmodule,Utils.netmodule
```

Resulting mental model:

```
Reference: Acme.Shop          (one assembly name / version / identity)
Files on disk:
  Acme.Shop.dll               (manifest lives here)
  Core.netmodule
  Utils.netmodule
```

Why this existed historically:

- download / load parts of a large assembly on demand
- mix languages into one assembly (e.g. C# module + VB module)
- keep one assembly boundary (`internal` shared) while splitting files

Why it is rare today:

- SDK-style projects (`dotnet build`) produce **one assembly file per project** by default
- NuGet, project references, and multi-project solutions replaced the need for multi-file assemblies
- you almost never create `.netmodule` files in modern app development

For this course: know the term **module** = “a file that can be part of an assembly”; assume **one project → one assembly → one file** unless you are reading old docs about multi-file assemblies.

2.3 The assembly manifest

The **manifest** is metadata describing the assembly as a whole.

It typically includes:

- **Assembly name** (simple name, e.g. `Acme.Shop`)
- **Version** (`Major.Minor.Build.Revision`)
- **Culture** (for satellite / localized assemblies; usually neutral)
- **Public key token** (if strongly named)
- **Referenced assemblies** (dependencies and their versions)
- **Exported types** / files belonging to the assembly
- Security / permission-related historical data (legacy .NET Framework)

Think of the manifest as the assembly's ID card and packing list.

```
Manifest says:
  Name:     Acme.Billing
  Version:  2.1.0.0
  Needs:    System.Text.Json 8.0.0
            Acme.Shared 1.4.0
```

2.4 How identity is formed

Assembly identity is more than the file name.

Classic identity parts:

1. Simple name
2. Version

3. Culture
4. Public key token (strong name)

Two DLLs both named `Utils.dll` can still be **different assemblies** if version or strong-name identity differs.

In modern .NET (Core / 5+), binding is primarily driven by:

- project references and NuGet restore
- runtime dependency graph (`deps.json`)
- optional binding redirects / framework rules on older .NET Framework apps

2.5 Inspecting an assembly

```
dotnet list reference

# IL / metadata inspection
ildasm MyLib.dll          # classic disassembler (Windows/.NET Framework tooling)
# or modern alternatives: ILSpy, dnSpy, dotPeek
```

```
using System.Reflection;

var asm = Assembly.GetExecutingAssembly();

Console.WriteLine(asm.FullName);
Console.WriteLine(asm.GetName().Name);
Console.WriteLine(asm.GetName().Version);

foreach (var name in asm.GetReferencedAssemblies())
{
    Console.WriteLine($"{name.Name} {name.Version}");
}
```

`Assembly.FullName` prints something like:

```
Acme.Shop, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
```

That string comes from manifest identity fields.

2.6 Types of assemblies (practical view)

- **Application assemblies** — entry point / host app (`Program` , web host, etc.)
- **Library assemblies** — reusable DLLs referenced by apps or other libraries
- **Satellite assemblies** — localized resources for a culture (`en-US` , `hy-AM` , …) — see below
- **Framework / runtime assemblies** — provided by the runtime / shared framework

Satellite assemblies (localization)

A **satellite assembly** is a small DLL that holds **translated resources** for one culture (language/region). It does **not** normally contain your app's business logic — only strings, images, or other resources for that language.

Why "satellite"? The main assembly is the "planet"; each language pack orbits it as a separate DLL.

Typical layout after build/publish:

```
MyApp/
  MyApp.dll                ← main assembly (neutral / default resources, usually Engli
  en-US/
    MyApp.resources.dll    ← satellite for American English
  hy-AM/
    MyApp.resources.dll    ← satellite for Armenian (Armenia)
  fr-FR/
    MyApp.resources.dll    ← satellite for French (France)
```

Important details:

Point	Meaning
Same simple name family	Often `MyApp.resources.dll` in a culture folder
Different **culture** in identity	Manifest culture is `en-US` , `hy-AM` , etc. (main app is usually `neutral`)
Same version as main (by convention)	Built to match the main assembly version
Loaded on demand	Runtime picks the satellite that matches the current UI culture

How the runtime chooses text:

```
Current UI culture = hy-AM
        │
        ▼
Look for hy-AM/MyApp.resources.dll  → found? use Armenian strings
        │ not found
        ▼
Fall back (e.g. hy) then to neutral resources inside MyApp.dll
```

Minimal idea in code (resource files):

```
Resources/
  Strings.resx           ← default (neutral) — "Hello"
  Strings.hy-AM.resx     ← Armenian       — "Բարև"
  Strings.fr-FR.resx     ← French         — "Bonjour"
```

```
using System.Globalization;
using System.Resources;

// Switch UI language
CultureInfo.CurrentUICulture = new CultureInfo("hy-AM");

var rm = new ResourceManager("MyApp.Resources.Strings", typeof(Program).Assembly);
Console.WriteLine(rm.GetString("Greeting"));  // Բարև  (from hy-AM satellite)
```

When you build, the compiler packs non-neutral `.resx` files into **satellite** DLLs under culture folders; the default `.resx` stays in the main assembly.

Why use satellites instead of one giant DLL?

1. Ship extra languages without rebuilding the whole app logic
2. Download/install only the languages you need
3. Translators can work on resource files, not on C# code
4. Keep the main assembly smaller

You mainly care about satellites when building **localized** desktop/web apps with `.resx` (or similar) resources. If the app is single-language, you typically never create them.

2.7 Single-file and published output

```
dotnet publish -c Release
```

You get an output folder with:

- your app assembly
- dependency DLLs (unless trimmed / single-file packed)
- `.deps.json` (dependency context)
- `.runtimeconfig.json` (runtime settings)

Single-file publish can pack many assemblies into one host executable, but logically they remain assemblies loaded by the runtime.

2.8 Why the manifest matters

The runtime uses it to:

1. **Identify** what was loaded
2. **Resolve dependencies** (which other assemblies are required)
3. **Version** components independently
4. **Support side-by-side** scenarios (especially historically with GAC / strong names)

2.9 Common misconceptions

Misconception	Reality
Namespace = assembly	No — one assembly can have many namespaces
File name = identity	File name helps humans; identity is name + version + culture + key
Manifest is a separate `.manifest` file you edit daily	Usually embedded metadata produced by the compiler
Every type is public outside the assembly	Only `public` types are visible to other assemblies (`internal` stays private)

```
public class VisibleOutsideAssembly { }
internal class OnlyInsideThisAssembly { }
```

`internal` is an **assembly** boundary, not a namespace boundary.

Takeaway: An assembly is the shippable unit of .NET code. Its manifest records identity and dependencies. Namespaces live *inside* assemblies; manifests describe the assembly itself.

3. Versioning, Private vs Shared Assemblies, Strong Names

3.1 Why assembly versioning exists

Without versioning:

- apps cannot express “I need Billing 2.x”
- two apps on one machine may break each other
- debugging “which DLL is actually loaded?” becomes painful

Versioning answers: **which exact build of a component am I talking about?**

3.2 Version numbers in .NET

```
Major.Minor.Build.Revision
Example: 2.4.1.0
```

Part	Typical meaning
Major	Breaking changes
Minor	New features, backward compatible
Build / Patch	Bug fixes
Revision	Hotfix / build counter

In modern SDK-style projects:

```
<PropertyGroup>
  <Version>2.4.1</Version>
  <AssemblyVersion>2.4.1.0</AssemblyVersion>
  <FileVersion>2.4.1.1234</FileVersion>
  <InformationalVersion>2.4.1+git.abc123</InformationalVersion>
</PropertyGroup>
```

Property	Used for
`AssemblyVersion`	Assembly identity / binding (important historically)
`FileVersion`	Windows file properties
`Version` / NuGet version	Package identity on NuGet
`InformationalVersion`	Human/diagnostic version (often includes commit)

NuGet uses **SemVer** for packages (`2.4.1` , `2.5.0-beta.1`), which is related but not identical to `AssemblyVersion` .

3.3 Private vs shared assemblies

Private assemblies

A **private assembly** is used by one application (or a specific app folder).

```
MyApp/
  MyApp.exe
  Acme.Billing.dll      ← private copy for this app
  Acme.Shared.dll
```

Characteristics:

- deployed with the app (bin / publish folder)
- simple to reason about: “this app has its own copies”
- different apps can use different versions side by side on disk
- default model for almost all modern .NET development

Shared assemblies

A **shared assembly** is intended for use by multiple applications from a common location.

Historically on .NET Framework this meant the **GAC** (Global Assembly Cache):

```
GAC
 └── Acme.Billing 1.0.0.0
 └── Acme.Billing 2.0.0.0   ← side-by-side versions possible
```

Characteristics:

- one install can serve many apps
- requires strong naming (on classic GAC)
- higher operational complexity (install, update, policy)
- powerful, but easy to get wrong

Modern reality (.NET Core / 5+)

There is **no GAC** in the classic sense for .NET Core+.

Sharing today usually means:

- **NuGet packages** restored per project / solution
- **shared framework** packs shipped with the runtime
- optional central package management in a repo

So “private by default, share via packages/framework” is the modern rule.

	Private	Shared (classic GAC)	Shared (modern)
Location	App folder	GAC	NuGet cache / shared framework
Strong name required?	No	Yes	Not for normal NuGet use
Side-by-side	Per-app folders	Versioned GAC entries	Per-restore graph / runtime packs
Typical use	App dependencies	Enterprise shared libs (legacy)	Packages & runtime

3.4 Strong names

What a strong name is

A **strong name** gives an assembly a cryptographically backed identity:

1. Simple name
2. Version
3. Culture
4. Public key (token) — from a key pair

```
Acme.Billing, Version=2.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089
```

`PublicKeyToken` is a short hash of the public key.

Why strong names were created

- **Unique identity** — reduce "wrong DLL with same file name" confusion
- **Integrity** — detect tampering (signature check)
- **GAC requirement** — shared cache needed strong names
- **Binding policy** — publisher policy / redirects keyed off strong identity

A strong name is **not** a full security sandbox by itself. It proves origin/integrity of that assembly identity; it does not automatically mean "safe code."

Creating and applying a strong name (.NET Framework style)

```
sn -k acme.snk
```

```
<PropertyGroup>
  <SignAssembly>true</SignAssembly>
  <AssemblyOriginatorKeyFile>acme.snk</AssemblyOriginatorKeyFile>
</PropertyGroup>
```

Strong names in modern .NET

You *can* still strong-name assemblies, but:

- most app development does **not** need it
- NuGet authenticity uses **package signing** / trusted sources
- strong naming is mainly relevant for legacy .NET Framework / GAC scenarios, or specific enterprise policies

Rule of thumb:

- Normal ASP.NET Core / worker / library app → private assemblies + NuGet, strong name optional
- Legacy Framework + GAC → strong names matter

3.5 Binding and version conflicts

Binding is the runtime process of answering:
"When code asks for assembly X version Y, which file on disk do I actually load?"

Conflicts happen when different parts of the app want **different versions** of the same assembly.

```
MyApp needs Acme.Shared 1.0
Plugin needs Acme.Shared 2.0
```

Why this is a problem

At runtime there is usually **one default load context** for the app. You cannot freely mix "type `Foo` from Acme.Shared 1.0" and "type `Foo` from Acme.Shared 2.0" as if they were the same type — they are different identities.

Possible outcomes:

Outcome	What happens
Unify to one version	NuGet/restore or redirects pick a single version for everyone
Binding redirect	Framework remaps "ask for 1.0" → "load 2.0"
Load failure	`FileNotFoundException` / `FileLoadException` / restore error
Both versions loaded	Possible with plugins / separate `AssemblyLoadContext` (advanced)

Classic case: .NET Framework binding redirects

On .NET Framework, the app often ships with one copy of a DLL, but compile-time references still ask for older versions. `app.config` / `Web.config` can redirect:

```
<runtime>
  <assemblyBinding xmlns="urn:schemas-microsoft-com:asm.v1">
    <dependentAssembly>
      <assemblyIdentity name="Newtonsoft.Json"
                        publicKeyToken="30ad4fe6b2a6aeed"
                        culture="neutral" />
      <bindingRedirect oldVersion="0.0.0.0-13.0.0.0"
                       newVersion="13.0.0.0" />
    </dependentAssembly>
  </assemblyBinding>
</runtime>
```

Meaning: “Anyone asking for Newtonsoft.Json from 0.0.0.0 through 13.0.0.0 should get 13.0.0.0.”

Visual Studio / NuGet often auto-adds these redirects when restoring packages on Framework projects.

Flow:

```
Library A was compiled against Json.NET 12.0
Library B was compiled against Json.NET 13.0
App folder contains Newtonsoft.Json 13.0.0.0
        |
        ▼
bindingRedirect → both A and B load 13.0
(works only if 13.0 is API-compatible enough for A)
```

Redirects fix **version mismatch**; they do **not** fix breaking API changes. If A needs an API removed in 13.0, redirect still blows up at runtime.

Modern case: .NET Core / .NET 5+ (NuGet graph)

There is no classic `bindingRedirect` -centric model for most apps. Instead, **NuGet restore** builds a dependency graph and picks versions **before** you run.

Example conflict during restore:

```
MyApp
 ├── Package.A 1.0  → needs Acme.Shared >= 1.0
 └── Package.B 2.0  → needs Acme.Shared >= 2.0
```

NuGet tries to select one version of `Acme.Shared` that satisfies all ranges (often the lowest version that works, with rules/updates over time). Result is recorded in:

- `project.assets.json` (under `obj/`)
- publish / runtimeconfig / deps files used at run time

If no single version can satisfy everyone, restore fails or warns — you must upgrade/downgrade packages until the graph is consistent.

Inspect:

```
dotnet list package --include-transitive
dotnet list package --outdated
```

Concrete conflict scenarios

1) Two NuGet packages want different major versions

```
App
 ├── Serilog.Sinks.File     → wants Serilog 3.x
 └── SomeOldPackage        → wants Serilog 2.x
```

What to do:

1. Upgrade `SomeOldPackage` if a newer release supports Serilog 3
2. Or replace the old package
3. Or (last resort) stay on older Serilog everywhere

Hope that “binding redirect magic” will merge Serilog 2 and 3 usually fails — majors often break APIs.

2) Transitive dependency surprise

You did not reference `Acme.Shared` directly, but two packages pull it transitively at different versions.

```
dotnet list package --include-transitive
```

Fix by adding an explicit reference to the version you want (force unify), or aligning package upgrades:

```
<PackageReference Include="Acme.Shared" Version="2.1.0" />
```

3) Plugin / add-in needs its own version — how to use `AssemblyLoadContext`

```
Host app     → Acme.Shared 1.0
Plugin.dll   → Acme.Shared 2.0
```

If both load into the **default** context, .NET will usually try to reuse one `Acme.Shared` and one side breaks.
Fix: load the plugin (and **its** copy of `Acme.Shared`) into a **separate** `AssemblyLoadContext` (**ALC** = Assembly Load Context — an isolated "bucket" where assemblies are loaded so different versions can coexist).

Rule that makes it work

Do **not** share `Acme.Shared` types across the host/plugin boundary when versions differ.

Instead, share a tiny **contract** assembly that **both** use at the **same** version:

```
Acme.Contracts.dll     ← IPlugin, DTOs made of primitive/shared-stable types (ONE versi
Host.exe               ← uses Acme.Shared 1.0 + Acme.Contracts
Plugin/
  Plugin.dll           ← uses Acme.Shared 2.0 + Acme.Contracts
  Acme.Shared.dll      ← 2.0 private copy next to the plugin


Host context:     Acme.Shared 1.0
Plugin context:   Acme.Shared 2.0
Both contexts:    Acme.Contracts (same)  ← only safe bridge
```

Step 1 — define the contract (shared, same version)

```
// Project: Acme.Contracts (net8.0 classlib)
namespace Acme.Contracts;

public interface IPlugin
{
    string Name { get; }
    string Execute(string input);  // keep boundary types simple (string, int, JSON…)
}
```

Host and plugin both reference `Acme.Contracts` — same package/version.

Step 2 — write the plugin

```
// Project: SamplePlugin
using Acme.Contracts;
using Acme.Shared; // 2.0 APIs — only used INSIDE the plugin

namespace SamplePlugin;

public sealed class UppercasePlugin : IPlugin
{
    public string Name => "Uppercase";

    public string Execute(string input)
    {
        // Fine to use Acme.Shared 2.0 here — stays inside plugin context
        var helper = new SharedHelperV2();
        return helper.Transform(input).ToUpperInvariant();
    }
}
```

Publish plugin to its own folder (include its dependency DLLs):

```
dotnet publish SamplePlugin -c Release -o ./plugins/SamplePlugin
```

```
plugins/SamplePlugin/
  SamplePlugin.dll
  Acme.Shared.dll          ← version 2.0
  Acme.Contracts.dll       ← same as host
```

Step 3 — custom load context (host)

Default `LoadFromAssemblyPath` is not enough: when the plugin asks for `Acme.Shared`, you must resolve it from the **plugin folder**, not from the host.

```
using System.Reflection;
using System.Runtime.Loader;

public sealed class PluginLoadContext : AssemblyLoadContext
{
    private readonly AssemblyDependencyResolver _resolver;

    public PluginLoadContext(string pluginPath)
        : base(isCollectible: true) // allows unload later
    {
        _resolver = new AssemblyDependencyResolver(pluginPath);
    }

    protected override Assembly? Load(AssemblyName assemblyName)
    {
        // Prefer assemblies sitting next to the plugin
        string? path = _resolver.ResolveAssemblyToPath(assemblyName);
        if (path is not null)
            return LoadFromAssemblyPath(path);

        // Not found in plugin folder → fall back to default context
        // (this is how Acme.Contracts can be shared with the host)
        return null;
    }
}
```

Step 4 — load, find `IPlugin`, call it

```
using System.Reflection;
using Acme.Contracts;

static IPlugin LoadPlugin(string pluginPath)
{
    var ctx = new PluginLoadContext(pluginPath);
    Assembly asm = ctx.LoadFromAssemblyPath(pluginPath);

    foreach (Type type in asm.GetTypes())
    {
        if (typeof(IPlugin).IsAssignableFrom(type) &&
            !type.IsInterface &&
            !type.IsAbstract)
        {
            return (IPlugin)Activator.CreateInstance(type)!;
        }
    }

    throw new InvalidOperationException($"No IPlugin in {pluginPath}");
}

// Usage
string path = Path.GetFullPath("plugins/SamplePlugin/SamplePlugin.dll");
IPlugin plugin = LoadPlugin(path);

Console.WriteLine(plugin.Name);
Console.WriteLine(plugin.Execute("hello"));  // crosses boundary only via IPlugin + str
```

Host can still use `Acme.Shared` 1.0 for its own code — that copy lives in the default context and does not have to match the plugin's 2.0.

Step 5 — unload (optional)

Because the context was `isCollectible: true`, you can use the plugin and then unload it. Important: drop all references to plugin instances/ `Type` / `Assembly` before (or right when) you unload.

```
using System.Reflection;
using Acme.Contracts;

WeakReference weakCtx = default!;

void LoadUseAndUnload(string pluginPath)
{
    var ctx = new PluginLoadContext(pluginPath);
    weakCtx = new WeakReference(ctx);

    Assembly asm = ctx.LoadFromAssemblyPath(pluginPath);

    // --- use plugin types (via the shared contract) ---
    IPlugin? plugin = null;

    foreach (Type type in asm.GetTypes())
    {
        if (typeof(IPlugin).IsAssignableFrom(type) &&
            !type.IsInterface &&
            !type.IsAbstract)
        {
            plugin = (IPlugin)Activator.CreateInstance(type)!;
            break;
        }
    }

    if (plugin is null)
        throw new InvalidOperationException($"No IPlugin in {pluginPath}");

    Console.WriteLine($"Loaded: {plugin.Name}");
    string result = plugin.Execute("hello");
    Console.WriteLine($"Result: {result}");

    // Clear strong refs so the collectible context can unload
    plugin = null;
    asm = null!;

    ctx.Unload();
}

string path = Path.GetFullPath("plugins/SamplePlugin/SamplePlugin.dll");
LoadUseAndUnload(path);
```

```
// Unload is async — GC must run before the context is fully gone
for (int i = 0; i < 10 && weakCtx.IsAlive; i++)
{
    GC.Collect();
    GC.WaitForPendingFinalizers();
}

Console.WriteLine(weakCtx.IsAlive
    ? "Context still alive (something still holds a reference)"
    : "Plugin context unloaded successfully");
```

Example console output:

```
Loaded: Uppercase
Result: HELLO
Plugin context unloaded successfully
```

Keep no static caches of plugin `Type` / `Assembly` / `IPlugin` instances if you want unload to succeed. A host field like `static IPlugin Current` would pin the context forever.

What you may / must not pass across the boundary

Safe to pass host ↔ plugin	Unsafe when Shared versions differ
`IPlugin` from `Acme.Contracts`	`Acme.Shared.Customer` (1.0 vs 2.0 are different types)
`string` , `int` , `bool` , `Guid`	Casting Shared 1.0 object to Shared 2.0 type
JSON / protobuf bytes	Returning a Shared type from `IPlugin`
Primitive DTOs defined in Contracts	Storing plugin types in host static fields (blocks unload)

Minimal end-to-end picture

```
Host (default ALC = default AssemblyLoadContext)
  ├── Acme.Shared 1.0
  ├── Acme.Contracts
  └── PluginLoadContext
        ├── SamplePlugin.dll
        ├── Acme.Shared 2.0     ← resolved from plugin folder
        └── uses Acme.Contracts from default ALC (shared)

Call path:
  host → IPlugin.Execute("hello") → plugin uses Shared 2.0 internally → returns string
```

When to use this vs unify versions

Situation	Prefer
Normal app + NuGet deps	Unify one version (simpler)
True plugin system, 3rd-party add-ins	`AssemblyLoadContext` + contracts
You control all code	Upgrade everyone to Shared 2.0

For most business apps you avoid this — keep one unified version. Use ALC plugins when isolation is a real product requirement.

4) “It works on my machine” wrong DLL in the folder

Publish/output folder accidentally contains an old `Acme.Shared.dll` while packages expected a newer one (manual copy, stale build, wrong working directory).

Fix: clean rebuild + publish from CI; don’t hand-copy DLLs into `bin`.

```
dotnet clean
dotnet build
dotnet publish -c Release
```

.NET Framework vs modern .NET — quick compare

	.NET Framework	.NET Core / 5+
When conflict is decided	Often at **runtime** (fusion / redirects)	Mostly at **restore/build** (NuGet graph)
Common fix	`bindingRedirect` in config	Align package versions
Config file	`app.config` / `Web.config`	`deps.json` / assets (generated)
Side-by-side versions	GAC + strong names; redirects	Separate apps, or `AssemblyLoadContext`

How to resolve conflicts in practice (checklist)

1. **See the graph** — `dotnet list package --include-transitive`
2. **Unify versions** — upgrade packages so everyone can use the same major
3. **Pin explicitly** — add a direct `PackageReference` to the chosen version
4. **Central Package Management** — one version list for the whole repo (`Directory.Packages.props`)
5. **Avoid hand-copied DLLs** — let restore/publish place assemblies
6. **Isolate only when you must** — plugins via `AssemblyLoadContext`, not for normal app deps
7. On legacy Framework — keep `bindingRedirect`s in sync (or enable auto-generation)

Mental model

```
Compile time:  "I was built against Acme.Shared 1.2"
Restore/build: "The app will ship Acme.Shared 2.0"   ← modern unification happens here
Runtime:       "Load the Acme.Shared we decided to ship"
```

Binding/version conflicts are less mysterious when you treat them as a **dependency graph problem** first, and a **runtime loading problem** second.

3.6 Practical recommendations

1. Prefer **private assemblies** deployed with the app.
2. Share code through **NuGet packages** (or project references in a monorepo), not GAC, for new work.

3. Version packages with **SemVer**; keep `AssemblyVersion` intentional if consumers bind to it.
4. Use strong names when policy/legacy requires them — not by default.
5. Treat “DLL Hell” as a packaging/process problem: automate publish and restore.

Scenario	Approach
Web API + internal helpers	Private DLLs in publish output
Reuse logging helpers across 5 services	Class library → NuGet (or project reference)
Old .NET Framework enterprise shared lib in GAC	Strong-named shared assembly
Two apps need different lib versions on one server	Private copies per app folder

Takeaway: Versioning identifies builds. Private assemblies are the default today. Classic shared (GAC) assemblies need strong names. Modern sharing is usually NuGet + runtime packs.

4. Class Libraries, .NET Standard vs .NET Core

4.1 What a class library is

A **class library** is a project that compiles to a reusable **DLL** (no required app entry point). Other projects reference it to share code.

Typical uses: domain models, data access helpers, shared DTOs/contracts, reusable utilities.

```
Acme.Shop.Api          (web app)
Acme.Shop.Worker       (background service)
        │
        ▼
Acme.Shop.Domain       (class library DLL)
Acme.Shop.Infrastructure (class library DLL)
```

```
dotnet new classlib -n Acme.Shop.Domain
dotnet add Acme.Shop.Api/Acme.Shop.Api.csproj reference Acme.Shop.Domain/Acme.Shop.Doma
```

4.2 Target frameworks (TFMs)

```
<TargetFramework>net8.0</TargetFramework>
```

Or multi-target:

```
<TargetFrameworks>net8.0;netstandard2.0</TargetFrameworks>
```

The TFM decides which APIs are available at compile time and which runtimes can consume the built assembly.

4.3 .NET Standard — what it was for

.NET Standard is a **formal API specification** (a contract), not a runtime you install and run apps on.

It answered: “Which APIs can I use so my library works on several .NET implementations?”

Historically: .NET Framework, .NET Core, Xamarin, Mono, Unity, UWP.

```
Your library targets .NET Standard 2.0
        │
        ├── runs on .NET Framework 4.6.1+
        ├── runs on .NET Core 2.0+
        ├── runs on modern .NET 5/6/7/8 (compatible)
        └── runs on other Standard-compatible platforms
```

Standard	Role
1.x	Narrow surface; wide but painful for library authors
2.0	Sweet spot for broad compatibility (Framework + Core)
2.1	More APIs; **not** supported by .NET Framework

4.4 .NET Core (and modern .NET) — what it is

.NET Core was the cross-platform, open-source redesign of .NET.
From **.NET 5** onward, branding unified as just **.NET** (`net5.0` , `net6.0` , `net8.0` , …).

Characteristics: runtime + SDK + libraries; side-by-side installs; cross-platform; ships BCL, ASP.NET Core, etc.

```
.NET Standard  = "API contract for libraries"
.NET Core/.NET = "actual platform that executes code"
```

4.5 Standard vs Core — side by side

	.NET Standard	.NET Core / modern .NET
Kind	Specification (API set)	Implementation / product
You run apps on it?	No	Yes
You write libraries for it?	Yes (historically)	Yes (`net8.0` class libs)
Goal	Portability across implementations	Build and run modern apps
Example TFM	`netstandard2.0`	`net8.0`

Analogy: **.NET Standard** ≈ USB specification; **.NET 8** ≈ a specific computer that implements specs and runs programs.

4.6 What should you target today?

Prefer modern TFMs for new libraries

If all consumers are .NET 6/8/9+:

```
<TargetFramework>net8.0</TargetFramework>
```

Use `netstandard2.0` when you must support .NET Framework

```
<TargetFramework>netstandard2.0</TargetFramework>
```

Trade-off: older API surface; some newer features unavailable unless you multi-target.

Multi-targeting when needed

```
<TargetFrameworks>netstandard2.0;net8.0</TargetFrameworks>
```

```
public static string Describe()
{
#if NET8_0_OR_GREATER
    return "Modern .NET APIs available";
#else
    return "Broad compatibility mode";
#endif
}
```

4.7 Class library design tips

1. Keep libraries focused — Domain vs Infrastructure vs Contracts.
2. Avoid framework-specific types in core libraries (e.g. don't put ASP.NET `HttpContext` in Domain).
3. Choose TFM from consumers, not from habit.
4. Prefer project references inside a solution; use NuGet when sharing across repos/teams.
5. Don't create a Standard library "just in case" if every consumer is already `net8.0` .

```
Do you need .NET Framework consumers?
  YES → netstandard2.0 (or multi-target)
  NO  → target net8.0 (or current LTS)
```

4.8 Apps vs libraries (TFM reminder)

Project	Typical TFM	Notes
ASP.NET Core API	`net8.0`	App host
Worker Service	`net8.0`	App host
Shared domain lib	`net8.0` or `netstandard2.0`	Depends on consumers
Legacy Framework app	`net48`	May reference Standard 2.0 libs

A `net8.0` app can reference `net8.0` and `netstandard2.0` libraries.

A `net48` app can reference `netstandard2.0` libraries, **not** a pure `net8.0` -only library.

Takeaway: Class libraries package reusable code as DLLs. .NET Standard is a portability contract. .NET Core / modern .NET is the platform you build and run on. For new work target `net8.0+` ; use Standard 2.0 only when you still need .NET Framework.

5. Settings and NuGet Packages

5.1 Two different “configuration” ideas

1. **App / runtime settings** — values your program reads (connection strings, feature flags, URLs).
2. **Project / package settings** — how the project builds and which NuGet packages it uses (`.csproj` , `NuGet.config`).

5.2 Application settings (modern .NET)

Configuration providers

```
Defaults
  → appsettings.json
  → appsettings.{Environment}.json
  → User secrets (dev)
  → Environment variables
  → Command-line args
```

Later sources override earlier ones.

Example `appsettings.json` :

```
{
  "ConnectionStrings": {
    "ShopDb": "Server=localhost;Database=Shop;Trusted_Connection=True"
  },
  "FeatureManagement": {
    "UseNewCheckout": true
  },
  "Shipping": {
    "DefaultCarrier": "DHL",
    "TimeoutSeconds": 30
  }
}
```

Reading configuration (manual `ConfigurationBuilder`)

```
string environment =
    Environment.GetEnvironmentVariable("DOTNET_ENVIRONMENT")
    ?? Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT")
    ?? "Production";

IConfiguration config = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)

    // optional: false → required file (throw if missing)
    .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)

    // optional: true → env file may be absent (e.g. no Staging file yet)
    .AddJsonFile($"appsettings.{environment}.json", optional: true, reloadOnChange: tru

    // OS environment variables (nested keys use __ )
    .AddEnvironmentVariables()

    // Command-line args (usually last → highest priority)
    .AddCommandLine(args)
    .Build();

ShippingOptions shipping = config.GetSection("Shipping").Get<ShippingOptions>();
```

ASP.NET Core hosts do the same layering for you via `WebApplication.CreateBuilder(args)`.

```
public class ShippingOptions
{
    public string DefaultCarrier { get; set; } = "";
    public int TimeoutSeconds { get; set; }
}
```

`optional: true` vs `false`

Setting	Meaning
`optional: false`	File **must** exist; missing file → exception at startup
`optional: true`	Missing file is OK; that layer is simply skipped

Typical pattern: base `appsettings.json` is required; `appsettings.{Environment}.json` is optional.

Environment-specific JSON

```
appsettings.json                ← shared defaults
appsettings.Development.json    ← local/dev overrides
appsettings.Production.json     ← production overrides
```

Pick the file with `DOTNET_ENVIRONMENT` / `ASPNETCORE_ENVIRONMENT`:

```
DOTNET_ENVIRONMENT=Development dotnet run --project 04.SettingsAndNuGet
DOTNET_ENVIRONMENT=Production  dotnet run --project 04.SettingsAndNuGet
```

Environment variables

Nested keys use **double underscore** `__` (because `:` is awkward in some shells):

```
# Overrides Shipping:DefaultCarrier
Shipping__DefaultCarrier=UPS DOTNET_ENVIRONMENT=Development \
  dotnet run --project 04.SettingsAndNuGet
```

Command-line args

Passed after `--` so `dotnet` does not eat them:

```
dotnet run --project 04.SettingsAndNuGet -- --Shipping:DefaultCarrier=DPD
# or
dotnet run --project 04.SettingsAndNuGet -- Shipping:DefaultCarrier=DPD
```

Command-line values override JSON and environment variables when registered last.

Secrets

```
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:ShopDb" "Server=.;Database=ShopDev;..."
```

For real deployments: environment variables, Key Vault, or cloud secret stores — **not** checked-in passwords.

Classic .NET Framework note

Older apps used `App.config` / `Web.config` (`appSettings` , `connectionStrings`). Modern .NET prefers `appsettings.json` + environment + Options pattern.

5.3 Project settings that affect packages

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>

    <PackageId>Acme.Shop.Domain</PackageId>
    <Version>1.2.0</Version>
    <Authors>Acme</Authors>
    <Description>Shop domain model</Description>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Newtonsoft.Json" Version="13.0.3" />
    <PackageReference Include="Serilog" Version="4.0.0" />
  </ItemGroup>
</Project>
```

`PackageReference` is the modern way to declare NuGet dependencies (replaces `packages.config` in most new projects).

5.4 What NuGet is

NuGet is the package manager for .NET. A `.nupkg` typically contains compiled assemblies for one or more TFMs, optional content/analyzers, and metadata (id, version, dependencies, license).

```
nuget.org (or private feed)
        │  restore
        ▼
local global cache (~/.nuget/packages)
        │  reference
        ▼
your project's compile + publish output
```

5.5 Everyday NuGet commands

```
dotnet add package Serilog.AspNetCore
dotnet add package Newtonsoft.Json --version 13.0.3
dotnet remove package Newtonsoft.Json
dotnet restore
dotnet list package
dotnet list package --outdated
dotnet pack -c Release
```

5.6 Package sources and `NuGet.config`

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <packageSources>
    <clear />
    <add key="nuget.org" value="https://api.nuget.org/v3/index.json" />
    <add key="acme-private" value="https://pkgs.example.com/acme/index.json" />
  </packageSources>
</configuration>
```

Place `NuGet.config` at **repo root** so restores are consistent for everyone and CI.

Can restore be inconsistent without it?

Yes. NuGet merges config from several places (machine → user → solution/repo → project folder). If the repo does **not** define sources clearly, each machine can behave differently.

Source of difference	What goes wrong
Developer A added a **private feed** in Visual Studio (user-level config)	A restores fine; teammate B and CI fail with "package not found"
Developer has an **extra/local feed** with a newer or modified package	A gets version/build X; B gets different bits from nuget.org
CI agent only has **nuget.org**	Green locally, red in pipeline
Someone disabled nuget.org only on their machine	Opposite problem: local fails, CI works
Package exists on private feed **and** nuget.org with same id/version but different content (rare but nasty)	Whichever feed wins first can differ by machine

So "inconsistent" usually means:

1. **Different feeds** → different ability to find packages
2. **Different winner** when the same package id exists on multiple feeds
3. **Works on my machine / fails in CI** (classic)

Repo-root `NuGet.config` with `<clear />` then an explicit source list fixes that: every clone and CI use the **same** feeds, not whatever happens to be installed on that laptop.

```
<packageSources>
  <clear />   <!-- ignore machine/user feeds for this repo -->
  <add key="nuget.org" value="https://api.nuget.org/v3/index.json" />
  <add key="acme-private" value="https://pkgs.example.com/acme/index.json" />
</packageSources>
```

Without `<clear />`, user-level feeds still get mixed in — so the file helps less.

Note: `NuGet.config` controls **where** packages are downloaded from. Package **versions** still come from `PackageReference` / Central Package Management. You need both for fully reproducible

restores.

Private feed (VPN) + nuget.org together

Very common setup:

- **nuget.org** — public packages (Newtonsoft.Json, Serilog, ASP.NET bits, …)
- **company private feed** — internal packages (`Acme.*`), only reachable on **VPN** / corporate network

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <packageSources>
    <clear />
    <add key="nuget.org" value="https://api.nuget.org/v3/index.json" />
    <add key="acme-private" value="https://pkgs.example.com/acme/index.json" />
  </packageSources>
</configuration>
```

How restore works with both:

```
dotnet restore
    │
    ├─ need Serilog 4.0        → found on nuget.org
    └─ need Acme.Shared 2.1    → found on acme-private (VPN required)
```

NuGet queries the configured sources and downloads each package from a feed that has it.

Practical rules:

Topic	Guidance
VPN off	Public packages may restore; **private** packages fail (“unable to load service index” / 401 / not found)
CI	Agents must reach the private feed (self-hosted on VPN, or cloud + feed credentials / service connection)
Credentials	Don’t put passwords in `NuGet.config` in git — use `dotnet nuget add source` with a stored token, credential provider, or CI secret

Topic	Guidance
Same package id on both feeds	Ambiguous — prefer unique internal ids (`Acme.Shared`) and never republish public names to private feed
Mapping (optional)	You can say “ `Acme.*` only from private” via `packageSourceMapping` so public feed isn’t even tried for internals

Optional **package source mapping** (recommended in larger orgs):

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <packageSources>
    <clear />
    <add key="nuget.org" value="https://api.nuget.org/v3/index.json" />
    <add key="acme-private" value="https://pkgs.example.com/acme/index.json" />
  </packageSources>

  <packageSourceMapping>
    <packageSource key="nuget.org">
      <package pattern="*" />
    </packageSource>
    <packageSource key="acme-private">
      <package pattern="Acme.*" />
    </packageSource>
  </packageSourceMapping>
</configuration>
```

Meaning:

- anything starting with `Acme.` → **only** private feed
- everything else → nuget.org

That avoids “wrong feed wins” and makes VPN-only internals explicit.

Auth example for local dev (token not committed):

```
dotnet nuget add source "https://pkgs.example.com/acme/index.json" \
  --name acme-private \
  --username "any" \
  --password "YOUR_PAT" \
  --store-password-in-clear-text   # or use a credential provider instead
```

Better: keep sources in repo `NuGet.config` (no secrets), and supply credentials via IDE login, Azure Artifacts Credential Provider, or CI variables.

Bottom line: yes — list **both** feeds. Connect VPN when restoring solutions that use internal packages; CI needs network + auth to that private feed too.

5.7 How restore and binding fit together

On `dotnet restore` / build:

1. Read `PackageReference` versions from projects
2. Resolve dependency graph (including transitive packages)
3. Download missing packages to cache
4. Write assets files (`project.assets.json`)
5. Compiler references the chosen assemblies

```
YourApp
  └── Package A 1.2
        └── Package B 3.1   ← transitive dependency
```

Use `dotnet list package --include-transitive` to inspect that graph.

5.8 Version ranges and good habits

Prefer pinned versions in apps:

```
<PackageReference Include="Serilog" Version="4.0.0" />
```

Good practices: restore in CI; don't commit `bin/` or package binaries; review major upgrades; prefer trusted packages; keep secrets out of git; use Central Package Management in monorepos.

```
<!-- Directory.Packages.props -->
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="Serilog" Version="4.0.0" />
  </ItemGroup>
</Project>


<!-- In each csproj -->
<PackageReference Include="Serilog" />
```